

Verificação formal de um software de controle de semáforos em tempo real

Igor Becker Boos - Jean Carlos Dapper

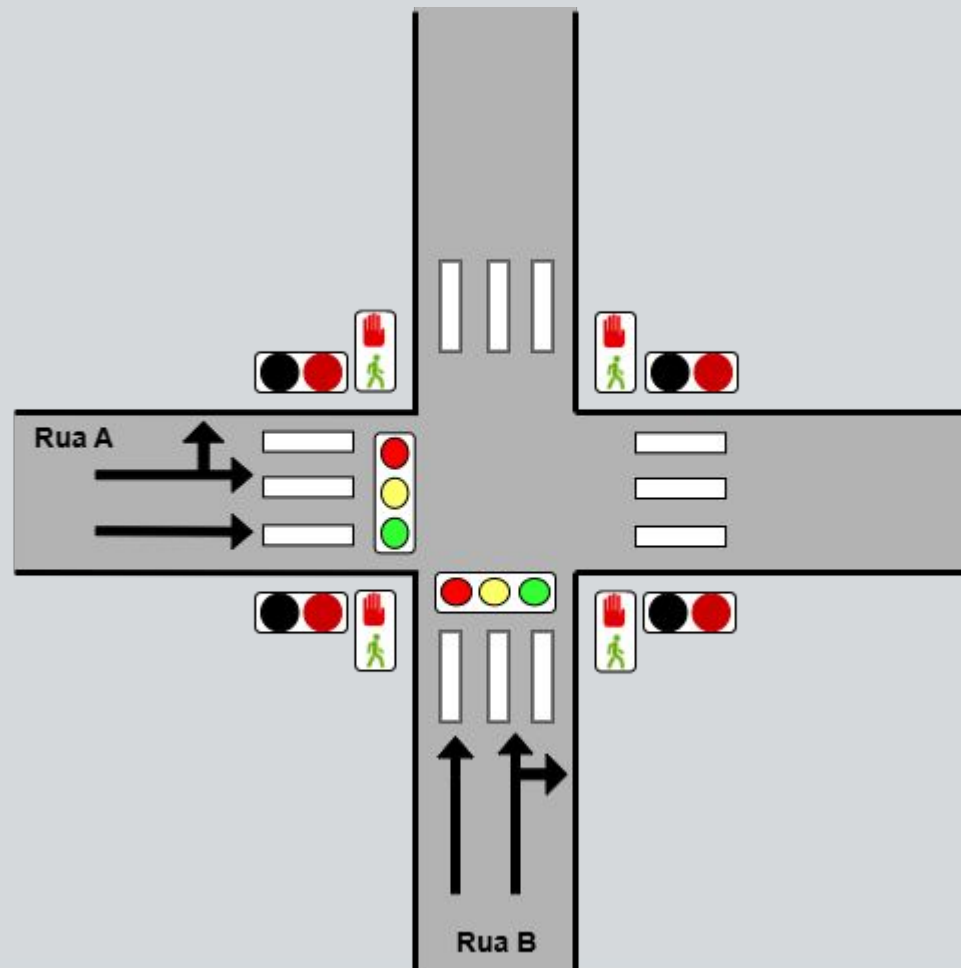
PPGESE | Joinville



INTRODUÇÃO

- Controle semafórico é um problema crítico em sistemas urbanos
- Sistemas embarcados exigem:
 - Confiabilidade
 - Determinismo
 - Segurança funcional
- Trabalho propõe:
 - Desenvolvimento do controlador em C
 - Testes unitários com Unity
 - Verificação formal com UPPAAL
- Objetivo:
 - Garantir corretude temporal e segurança do sistema

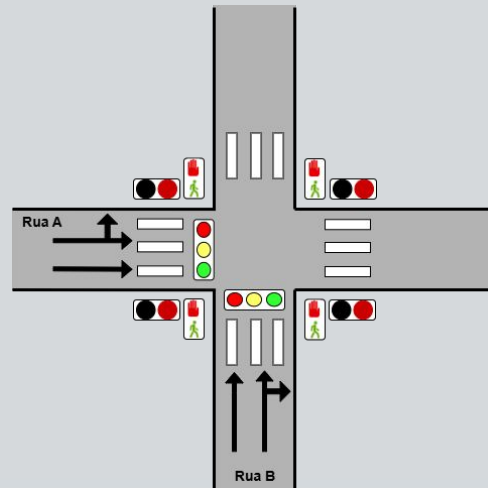
ESPECIFICAÇÃO DO SISTEMA



ESPECIFICAÇÃO DO SISTEMA

Características do sistema

- Cruzamento entre Rua A e Rua B
- Dois semáforos veiculares
- Um semáforo de pedestres
- Máquina de estados temporizada
- Alternância automática:
 - Verde
 - Amarelo
 - Vermelho



Funcionalidades adicionais

- Solicitação de travessia de pedestres
- Modo emergência/manutenção
- Estados seguros com “todos vermelhos”

REQUISITOS FUNCIONAIS

ID	Requisito Funcional
RF-01	O sistema deve controlar 3 semáforos independentes, sendo um associado à Rua A, outro associado à Rua B, outro associado ao sinaleiro dos pedestres.
RF-02	O sistema deve iniciar sua operação com todos os sinaleiros em vermelho.
RF-03	O sistema deve alternar automaticamente o direito de passagem entre Rua A e Rua B.
RF-04	O sistema deve garantir que apenas uma rua possua sinal verde por vez.
RF-05	O sistema deve possuir para os sinaleiros associados a rua A e B um estado amarelo antes da troca do semáforo para vermelho.
RF-06	O estado amarelo deve permanecer ativo durante um intervalo de tempo configurado.
RF-07	Após o estado amarelo, o sistema deve colocar ambos os semáforos em vermelho durante um intervalo de segurança.
RF-08	O sistema deve permitir que veículos provenientes da Rua A continuem na Rua A ou realizem conversão para a Rua B quando o semáforo da Rua A estiver verde.
RF-09	O sistema deve permitir que veículos provenientes da Rua B continuem na Rua B ou realizem conversão para a Rua A quando o semáforo da Rua B estiver verde.
RF-10	O sistema deve possuir um botão de solicitação de travessia de pedestres localizados nos cantos do cruzamento.
RF-11	Quando um botão de pedestre for pressionado, o sistema deve registrar a solicitação de travessia.
RF-12	O sistema deve atender solicitações de travessia somente após o encerramento seguro do ciclo atual de veículos.

RF-13	Durante a travessia de pedestres, ambos os semáforos de veículos devem permanecer em vermelho.
RF-14	O sistema deve liberar a travessia de pedestres durante um intervalo de tempo pré-definido.
RF-15	Após o término da travessia de pedestres, o sistema deve retornar automaticamente ao controle normal do tráfego.
RF-16	O sistema deve possuir um botão dedicado ao acionamento do modo emergência/manutenção.
RF-17	O sistema deve concluir a transição segura do estado atual antes de entrar em emergência.
RF-18	Durante o modo emergência/manutenção, todos os semáforos devem permanecer em vermelho.
RF-19	Durante o modo emergência/manutenção, nenhuma transição automática de estados deve ocorrer.
RF-20	Quando o modo emergência/manutenção for desativado, o sistema deve reiniciar sua operação a partir do estado inicial configurado.
RF-21	O sistema deve operar em execução cíclica infinita.
RF-22	O sistema deve utilizar temporizadores internos para controle das transições de estados.
RF-23	O sistema deve garantir que, após uma travessia de pedestres, pelo menos um ciclo de liberação da Rua A e um ciclo de liberação da Rua B ocorram antes que uma nova solicitação de travessia de pedestres seja atendida.

REQUISITOS FUNCIONAIS

Principais requisitos

- Apenas uma rua pode ter verde por vez
- Estado amarelo obrigatório antes do vermelho
- Intervalo de segurança “all red”
- Atendimento seguro de pedestres
- Emergência interrompe operação automática
- Sistema deve operar infinitamente sem deadlock

Requisitos importantes

- RF-04 → exclusão mútua dos verdes
- RF-13 → pedestres somente com veículos parados
- RF-23 → fairness entre Rua A e Rua B

SOFTWARE DO CONTROLADOR

Implementação

- Linguagem C
- Máquina de estados finita temporizada


```

int main(void)
{
    changeState(STATE_INIT);
    while(1U)
    {
        // EVENTOS
        if(gTick == 15U)
        {
            pedestrianButtonPressed();
        }
        if(gTick == 20U)
        {
            pedestrianButtonPressed();
        }
        if(gTick == 40U)
        {
            emergencyButtonPressed();
        }
        if(gTick == 70U)
        {
            emergencyButtonReleased();
        }

        // FSM
        trafficControllerTick();

        // DEBUG
        printSystemState();

        // TICK
        gTick++;
        if(gTick > 100U) // Simulação limitada.
        {
            break;
        }
    }
    return 0U;
}

```

T= 0	ST= 0	A=RED	B=RED	PED=RED
T= 1	ST= 1	A=GREEN	B=RED	PED=RED
T= 2	ST= 1	A=GREEN	B=RED	PED=RED
T= 3	ST= 1	A=GREEN	B=RED	PED=RED
T= 4	ST= 1	A=GREEN	B=RED	PED=RED
T= 5	ST= 1	A=GREEN	B=RED	PED=RED
T= 6	ST= 1	A=GREEN	B=RED	PED=RED
T= 7	ST= 1	A=GREEN	B=RED	PED=RED
T= 8	ST= 1	A=GREEN	B=RED	PED=RED
T= 9	ST= 1	A=GREEN	B=RED	PED=RED
T= 10	ST= 1	A=GREEN	B=RED	PED=RED
T= 11	ST= 2	A=YELLOW	B=RED	PED=RED
T= 12	ST= 2	A=YELLOW	B=RED	PED=RED
T= 13	ST= 2	A=YELLOW	B=RED	PED=RED
T= 14	ST= 3	A=RED	B=RED	PED=RED
[EVENT] Pedestrian button pressed				
T= 15	ST= 3	A=RED	B=RED	PED=RED
T= 16	ST= 7	A=RED	B=RED	PED=GREEN
T= 17	ST= 7	A=RED	B=RED	PED=GREEN
T= 18	ST= 7	A=RED	B=RED	PED=GREEN
T= 19	ST= 7	A=RED	B=RED	PED=GREEN
[EVENT] Pedestrian button pressed				
T= 20	ST= 7	A=RED	B=RED	PED=GREEN
T= 21	ST= 7	A=RED	B=RED	PED=GREEN
T= 22	ST= 7	A=RED	B=RED	PED=GREEN
T= 23	ST= 7	A=RED	B=RED	PED=GREEN
T= 24	ST= 1	A=GREEN	B=RED	PED=RED
T= 25	ST= 1	A=GREEN	B=RED	PED=RED

T= 26	ST= 1	A=GREEN	B=RED	PED=RED
T= 27	ST= 1	A=GREEN	B=RED	PED=RED
T= 28	ST= 1	A=GREEN	B=RED	PED=RED
T= 29	ST= 1	A=GREEN	B=RED	PED=RED
T= 30	ST= 1	A=GREEN	B=RED	PED=RED
T= 31	ST= 1	A=GREEN	B=RED	PED=RED
T= 32	ST= 1	A=GREEN	B=RED	PED=RED
T= 33	ST= 1	A=GREEN	B=RED	PED=RED
T= 34	ST= 2	A=YELLOW	B=RED	PED=RED
T= 35	ST= 2	A=YELLOW	B=RED	PED=RED
T= 36	ST= 2	A=YELLOW	B=RED	PED=RED
T= 37	ST= 3	A=RED	B=RED	PED=RED
T= 38	ST= 3	A=RED	B=RED	PED=RED
T= 39	ST= 4	A=RED	B=GREEN	PED=RED
[EVENT] Emergency requested				
T= 40	ST= 4	A=RED	B=GREEN	PED=RED
T= 41	ST= 4	A=RED	B=GREEN	PED=RED
T= 42	ST= 4	A=RED	B=GREEN	PED=RED
T= 43	ST= 4	A=RED	B=GREEN	PED=RED
T= 44	ST= 4	A=RED	B=GREEN	PED=RED
T= 45	ST= 4	A=RED	B=GREEN	PED=RED
T= 46	ST= 4	A=RED	B=GREEN	PED=RED
T= 47	ST= 4	A=RED	B=GREEN	PED=RED
T= 48	ST= 4	A=RED	B=GREEN	PED=RED
T= 49	ST= 5	A=RED	B=YELLOW	PED=RED
T= 50	ST= 5	A=RED	B=YELLOW	PED=RED

T= 51	ST= 5	A=RED	B=YELLOW	PED=RED
T= 52	ST= 6	A=RED	B=RED	PED=RED
T= 53	ST= 6	A=RED	B=RED	PED=RED
T= 54	ST= 8	A=RED	B=RED	PED=RED
T= 55	ST= 8	A=RED	B=RED	PED=RED
T= 56	ST= 8	A=RED	B=RED	PED=RED
T= 57	ST= 8	A=RED	B=RED	PED=RED
T= 58	ST= 8	A=RED	B=RED	PED=RED
T= 59	ST= 8	A=RED	B=RED	PED=RED
T= 60	ST= 8	A=RED	B=RED	PED=RED
T= 61	ST= 8	A=RED	B=RED	PED=RED
T= 62	ST= 8	A=RED	B=RED	PED=RED
T= 63	ST= 8	A=RED	B=RED	PED=RED
T= 64	ST= 8	A=RED	B=RED	PED=RED
T= 65	ST= 8	A=RED	B=RED	PED=RED
T= 66	ST= 8	A=RED	B=RED	PED=RED
T= 67	ST= 8	A=RED	B=RED	PED=RED
T= 68	ST= 8	A=RED	B=RED	PED=RED
T= 69	ST= 8	A=RED	B=RED	PED=RED
[EVENT] Emergency released				
T= 70	ST= 0	A=RED	B=RED	PED=RED
T= 71	ST= 1	A=GREEN	B=RED	PED=RED
T= 72	ST= 1	A=GREEN	B=RED	PED=RED
T= 73	ST= 1	A=GREEN	B=RED	PED=RED
T= 74	ST= 1	A=GREEN	B=RED	PED=RED
T= 75	ST= 1	A=GREEN	B=RED	PED=RED

T= 76	ST= 1	A=GREEN	B=RED	PED=RED
T= 77	ST= 1	A=GREEN	B=RED	PED=RED
T= 78	ST= 1	A=GREEN	B=RED	PED=RED
T= 79	ST= 1	A=GREEN	B=RED	PED=RED
T= 80	ST= 1	A=GREEN	B=RED	PED=RED
T= 81	ST= 2	A=YELLOW	B=RED	PED=RED
T= 82	ST= 2	A=YELLOW	B=RED	PED=RED
T= 83	ST= 2	A=YELLOW	B=RED	PED=RED
T= 84	ST= 3	A=RED	B=RED	PED=RED
T= 85	ST= 3	A=RED	B=RED	PED=RED
T= 86	ST= 7	A=RED	B=RED	PED=GREEN
T= 87	ST= 7	A=RED	B=RED	PED=GREEN
T= 88	ST= 7	A=RED	B=RED	PED=GREEN
T= 89	ST= 7	A=RED	B=RED	PED=GREEN
T= 90	ST= 7	A=RED	B=RED	PED=GREEN
T= 91	ST= 7	A=RED	B=RED	PED=GREEN
T= 92	ST= 7	A=RED	B=RED	PED=GREEN
T= 93	ST= 7	A=RED	B=RED	PED=GREEN
T= 94	ST= 1	A=GREEN	B=RED	PED=RED
T= 95	ST= 1	A=GREEN	B=RED	PED=RED
T= 96	ST= 1	A=GREEN	B=RED	PED=RED
T= 97	ST= 1	A=GREEN	B=RED	PED=RED
T= 98	ST= 1	A=GREEN	B=RED	PED=RED
T= 99	ST= 1	A=GREEN	B=RED	PED=RED
T=100	ST= 1	A=GREEN	B=RED	PED=RED

TESTES UNITÁRIOS

Estratégia de testes

- Biblioteca Unity
- Classes de equivalência
- Validação individual das funções

Resultados

- 44 casos de teste implementados
- 0 falhas
- 0 testes ignorados

TESTES UNITÁRIOS

Exemplo função updateOutputs()

```
static void updateOutputs(void)
{
    switch(gState)
    {
        case STATE_INIT:
            gLights.roadA = LIGHT_RED;
            gLights.roadB = LIGHT_RED;
            gLights.pedestrian = PED_RED;

            break;

        case STATE_A_GREEN:
            gLights.roadA = LIGHT_GREEN;
            gLights.roadB = LIGHT_RED;
            gLights.pedestrian = PED_RED;

            break;

        case STATE_A_YELLOW:
            gLights.roadA = LIGHT_YELLOW;
            gLights.roadB = LIGHT_RED;
            gLights.pedestrian = PED_RED;

            break;

        case STATE_B_GREEN:
            gLights.roadA = LIGHT_RED;
            gLights.roadB = LIGHT_GREEN;
            gLights.pedestrian = PED_RED;

            break;

        case STATE_B_YELLOW:
            gLights.roadA = LIGHT_RED;
            gLights.roadB = LIGHT_YELLOW;
            gLights.pedestrian = PED_RED;

            break;
    }
}
```

```
case STATE_ALL_RED_TO_A:
case STATE_ALL_RED_TO_B:
    gLights.roadA = LIGHT_RED;
    gLights.roadB = LIGHT_RED;
    gLights.pedestrian = PED_RED;

    break;

case STATE_PEDESTRIAN:
    gLights.roadA = LIGHT_RED;
    gLights.roadB = LIGHT_RED;
    gLights.pedestrian = PED_GREEN;

    break;

case STATE_EMERGENCY:
    gLights.roadA = LIGHT_RED;
    gLights.roadB = LIGHT_RED;
    gLights.pedestrian = PED_RED;

    break;

default:
    gLights.roadA = LIGHT_RED;
    gLights.roadB = LIGHT_RED;
    gLights.pedestrian = PED_RED;

    break;
}
```

TESTES UNITÁRIOS

Exemplo função updateOutputs()

```
void test_updateOutputs_STATE_A_GREEN(void)
{
    unsigned int targetTicks = TIME_ALL_RED;

    runTicks(targetTicks);

    TEST_ASSERT_EQUAL(
        LIGHT_GREEN,
        getRoadA()
    );

    TEST_ASSERT_EQUAL(
        LIGHT_RED,
        getRoadB()
    );

    TEST_ASSERT_EQUAL(
        PED_RED,
        getPedestrian()
    );
}
```

TESTES UNITÁRIOS

Exemplo função updateOutputs()

Tabela II: Classes de equivalência da função updateOutputs

Classe	Descrição	Domínio da entrada
C1	Estado inicial / tudo vermelho	gState = 0
C2	Rua A verde	gState = 1
C3	Rua A amarelo	gState = 2
C4	Rua B verde	gState = 4
C5	Rua B amarelo	gState = 5
C6	Todos vermelho	gState = 3 ou 6
C7	Pedestre verde	gState = 7
C8	Emergência	gState = 8
C9	Estado inválido	gState < 0 ou gState > 8

TESTES UNITÁRIOS

Exemplo função updateOutputs()

Tabela III: Casos de teste da função updateOutputs

Classe	Entrada (gState)	Saída Esperada
C1	0	A=RED, B=RED, PED=RED
C2	1	A=GREEN, B=RED, PED=RED
C3	2	A=YELLOW, B=RED, PED=RED
C4	4	A=RED, B=GREEN, PED=RED
C5	5	A=RED, B=YELLOW, PED=RED
C6	3	A=RED, B=RED, PED=RED
C6	6	A=RED, B=RED, PED=RED
C7	7	A=RED, B=RED, PED=GREEN
C8	8	A=RED, B=RED, PED=RED
C9	9	A=RED, B=RED, PED=RED

TESTES UNITÁRIOS

Exemplo função updateOutputs()

Tabela IV: Resultados dos testes da função updateOutputs

#	Teste Executado	Resultado
1	test_updateOutputs_STATE_INIT	PASS
2	test_updateOutputs_STATE_A_GREEN	PASS
3	test_updateOutputs_STATE_A_YELLOW	PASS
4	test_updateOutputs_STATE_B_GREEN	PASS
5	test_updateOutputs_STATE_B_YELLOW	PASS
6	test_updateOutputs_ALL_RED_TO_B	PASS
7	test_updateOutputs_ALL_RED_TO_A	PASS
8	test_updateOutputs_PEDESTRIAN	PASS
9	test_updateOutputs_EMERGENCY	PASS
10	test_updateOutputs_INVALID	PASS
Total de testes		10
Falhas		0
Ignorados		0

MODELAGEM DO SISTEMA

Ferramenta utilizada

- UPPAAL
- Autômatos temporizados

Modelos criados

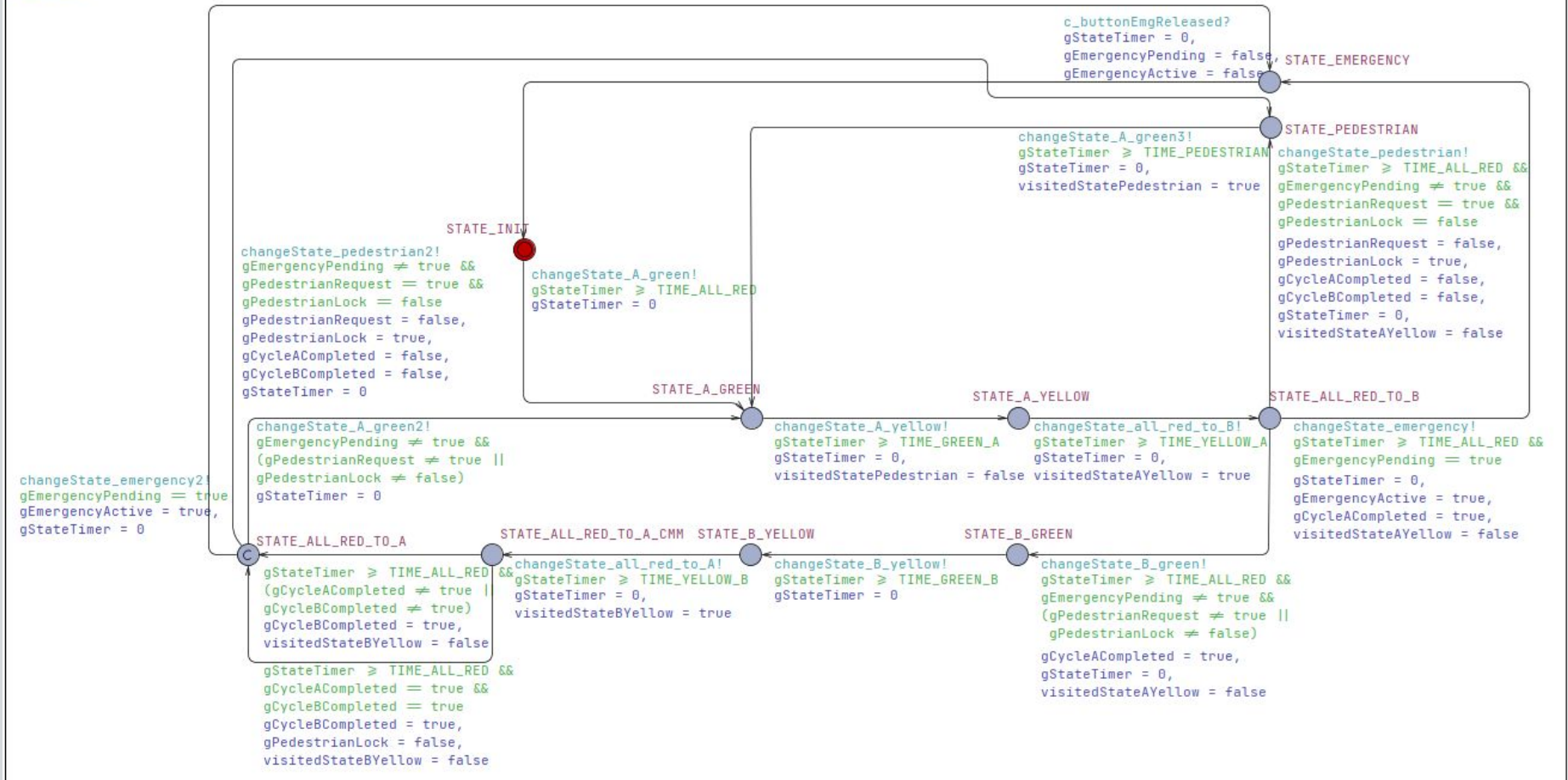
- Controlador
- Semáforo A
- Semáforo B
- Semáforo Pedestre
- Botão de pedestre
- Botão de emergência

Características

- Sincronização via broadcast channels
- Relógios temporizados
- Representação fiel ao código em C

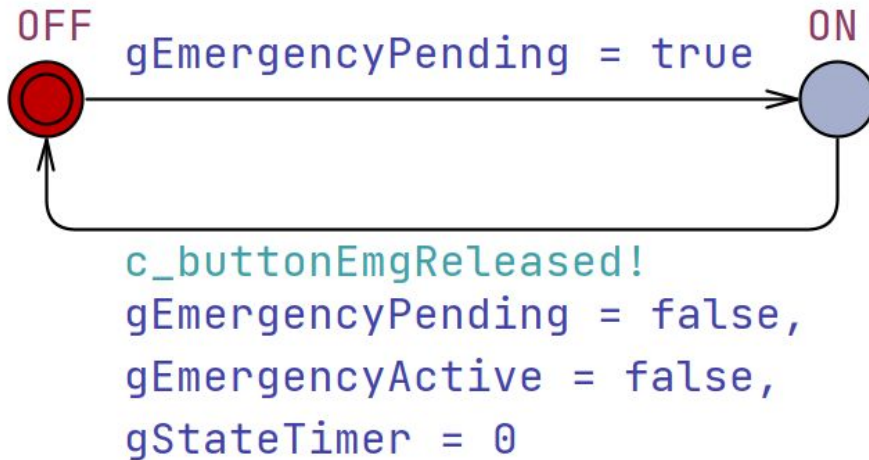
MODELAGEM DO SISTEMA

Controlador

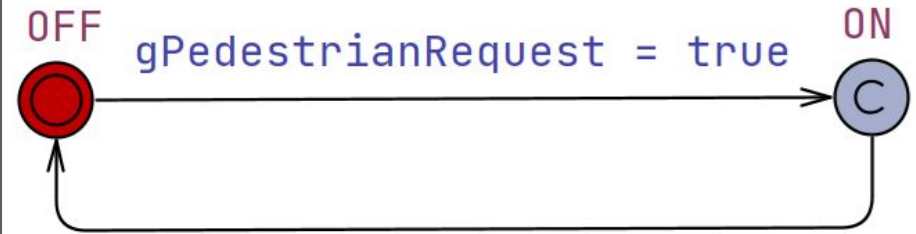


MODELAGEM DO SISTEMA

Botao_emergencia

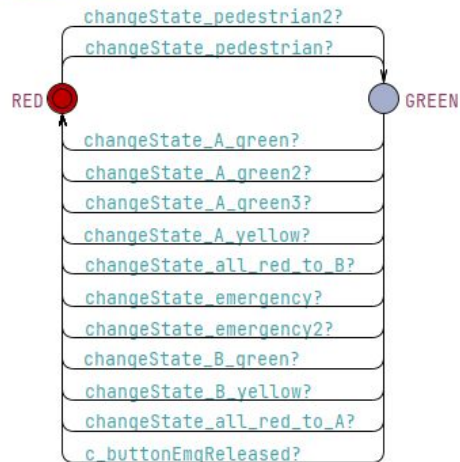


Botao_pedestre

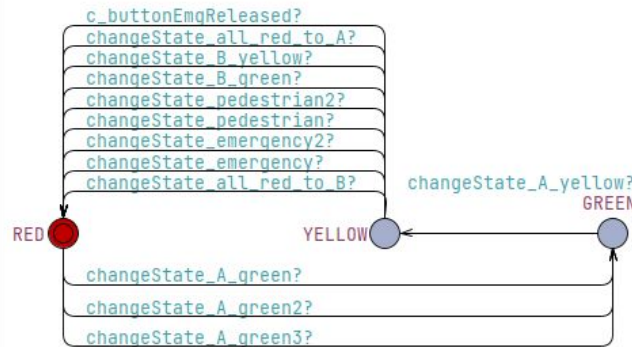


MODELAGEM DO SISTEMA

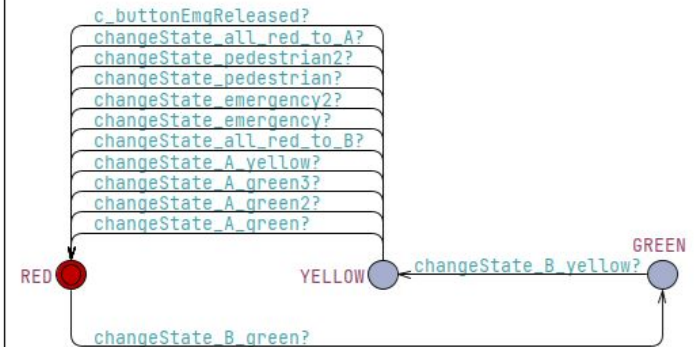
Semaforo_pedestre



Semaforo_A



Semaforo_B



VERIFICAÇÃO FORMAL DO MODELO

```
/* RF-01A */ E⇨ Semaforo_A.RED
/* RF-01B */ E⇨ Semaforo_A.GREEN
/* RF-01C */ E⇨ Semaforo_A.YELLOW
/* RF-01D */ E⇨ Semaforo_B.RED
/* RF-01E */ E⇨ Semaforo_B.GREEN
/* RF-01F */ E⇨ Semaforo_B.YELLOW
/* RF-01G */ E⇨ Semaforo_pedestre.RED
/* RF-01H */ E⇨ Semaforo_pedestre.GREEN
/* RF-02 */ A[] ( Controlador.STATE_INIT imply Semaforo_A.RED && Semaforo_B.RED && Semaforo_pedestre.RED )
/* RF-03A */ E⇨ Controlador.STATE_A_GREEN imply not Controlador.STATE_B_GREEN
/* RF-03B */ E⇨ Controlador.STATE_B_GREEN imply not Controlador.STATE_A_GREEN
/* RF-04A */ A[] not (Semaforo_A.GREEN && Semaforo_B.GREEN)
/* RF-04B */ A[] not (Semaforo_A.GREEN && Semaforo_B.GREEN && Semaforo_pedestre.GREEN)
/* RF-05A */ A[] (Controlador.STATE_ALL_RED_TO_B imply visitedStateAYellow = true)
/* RF-05B */ A[] (Controlador.STATE_ALL_RED_TO_A_CMM imply visitedStateBYellow = true)
/* RF-06A */ A[] ( Controlador.STATE_A_YELLOW and gStateTimer < TIME_YELLOW_A imply not Controlador.STATE_ALL_RED_TO_B )
/* RF-06B */ A[] ( Controlador.STATE_B_YELLOW && gStateTimer < TIME_YELLOW_B imply not Controlador.STATE_ALL_RED_TO_A_CMM )
/* RF-07A */ A[] (Controlador.STATE_ALL_RED_TO_B imply Semaforo_A.RED && Semaforo_B.RED)
/* RF-07B */ A[] (Controlador.STATE_ALL_RED_TO_A_CMM imply Semaforo_A.RED && Semaforo_B.RED)
/* RF-10A */ E⇨ Botao_pedestre.OFF
/* RF-10B */ E⇨ Botao_pedestre.ON
/* RF-11 */ A[] ( Botao_pedestre.ON imply gPedestrianRequest = true )
/* RF-12 */ A[] (Semaforo_pedestre.GREEN imply (Semaforo_A.RED && Semaforo_B.RED))
/* RF-13 */ A[] (Semaforo_pedestre.GREEN imply (Semaforo_A.RED && Semaforo_B.RED))
/* RF-15 */ A[] ((Controlador.STATE_A_GREEN && visitedStatePedestrian = true) imply visitedStatePedestrian = true)
/* RF-16A */ E⇨ Botao_emergencia.OFF
/* RF-16B */ E⇨ Botao_emergencia.ON
/* RF-17 */ A[] (gEmergencyActive imply Controlador.STATE_EMERGENCY)
/* RF-18 */ A[] (Controlador.STATE_EMERGENCY imply (Semaforo_A.RED && Semaforo_B.RED && Semaforo_pedestre.RED))
/* RF-19 */ A[] (Controlador.STATE_EMERGENCY imply not Controlador.STATE_A_GREEN)
/* RF-20 */ (not gEmergencyActive && Controlador.STATE_EMERGENCY) → Controlador.STATE_INIT
/* RF-21 */ A[] not deadlock
/* RF-23 */ A[] Controlador.STATE_PEDESTRIAN imply (gCycleACompleted = true && gCycleBCompleted = true)
```


Obrigado

